

第四讲： 神经网络如何学习

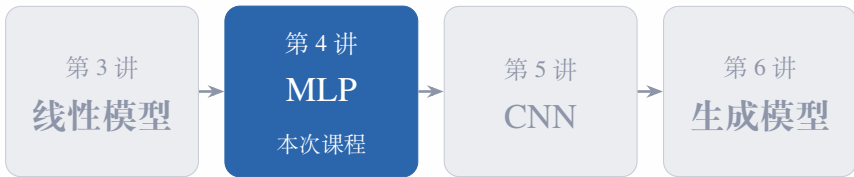
从线性模型到多层感知机

廖振宇

电子信息与通信学院
华中科技大学
2026 年秋季学期



本讲在课程中的位置



非线性表示、反向传播、训练与泛化
本讲建立从线性分类模型到深层视觉模型的桥梁

本讲内容

01

从线性模型到多层感知机

02

用反向传播训练多层感知机

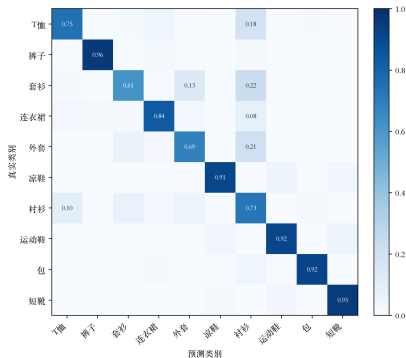
03

模型的训练与泛化

1

从线性模型到多层感知机

回顾：线性基线犯了哪些错误？



来源: <https://github.com/zalandoresearch/fashion-mnist>

第三讲中的广义线性基线

测试准确率约 **82.8%**

- 套衫、外套与衬衫相互混淆
- 运动鞋与短靴易相互混淆
- 相近轮廓需要组合多个局部证据

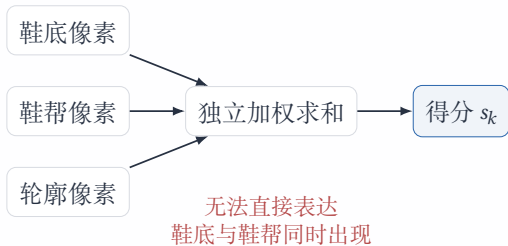
思考

保持（广义）线性模型，继续增加模型参数，能解决这些错误吗？

线性类别得分

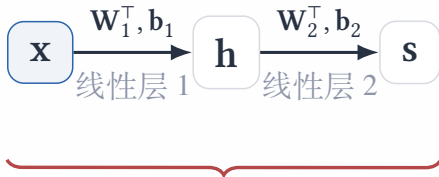
$$s_k = \mathbf{w}_k^\top \mathbf{x} + b_k = \sum_{j=1}^d w_{jk} x_j + b_k$$

- 每个像素先**独立**贡献得分
- 所有贡献只经过一次加权求和
- 边界始终是一个超**平面**



两个线性层？仍然会“合并”为一个线性层

$$\begin{aligned} \mathbf{h} &= \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1 \\ \mathbf{s} &= \mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2 \\ &= \underbrace{\mathbf{W}_2^\top \mathbf{W}_1^\top}_{\mathbf{W}_{\text{eff}}^\top} \mathbf{x} + \underbrace{\mathbf{W}_2^\top \mathbf{b}_1 + \mathbf{b}_2}_{\mathbf{b}_{\text{eff}}} \end{aligned}$$



结论

两个连续线性层仍然等价于一个线性变换

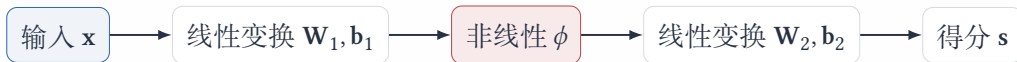
一个等效线性层

1 激活函数：复合函数带来非线性

定义 | 激活函数

激活函数作用于线性变换结果，可以带来非线性

$$\mathbf{z} = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1 \quad \mathbf{h} = \phi(\mathbf{z})$$

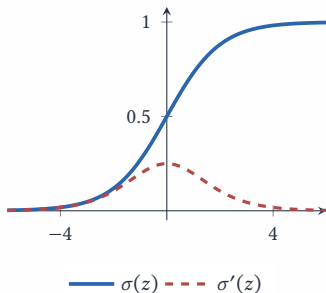


结论

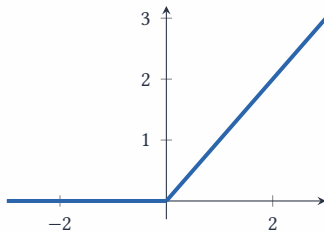
参数数量可以增加容量，非线性激活改变函数类型（线性 $\Rightarrow$ 非线性函数）

激活函数：从 Sigmoid 的饱和区到 ReLU

Sigmoid 与导数



ReLU



结论

- Sigmoid 函数在饱和区的梯度接近 0
- ReLU 在正区间保留梯度 ($= 1$): 利于 (基于梯度的) 模型训练

单隐藏层多层感知机怎样计算类别得分？



模型结构

输入图像 $\mathbb{R}^{784} \Rightarrow$ 隐藏层宽度（神经元个数） $\mathbb{R}^r \Rightarrow$ 输出类别得分 $\mathbb{R}^{10}$

定义 | 多层感知机

多层感知机（Multilayer Perceptron, MLP）由线性层与非线性激活函数复合而成
本讲先研究单隐藏层 MLP

1 一张图说清楚前向传播



$$\mathbf{z} = \mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1 \quad \in \mathbb{R}^r$$

$$\mathbf{h} = \text{ReLU}(\mathbf{z}) \quad \in \mathbb{R}^r$$

$$\mathbf{s} = \mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2 \quad \in \mathbb{R}^K$$

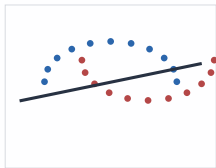
$$\mathbf{p} = \text{Softmax}(\mathbf{s}) \quad \in \mathbb{R}^K$$

$$L = -\mathbf{y}^\top \log \mathbf{p} \quad \in \mathbb{R}$$

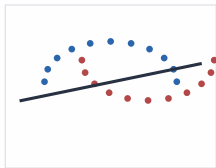
$$= -\sum_{k=1}^K y_k \log p_k = -\log p_y$$

结论

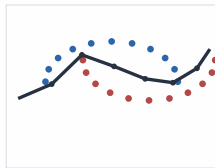
- 箭头：从左到右的**前向**计算
- $\mathbf{y} \in \{0, 1\}^K$: one-hot 标签向量
- 输入 $\Rightarrow$ 表示 $\Rightarrow$ 类别得分 $\Rightarrow$ 概率与损失



线性 Softmax



双线性层+Softmax



ReLU MLP

结论

ReLU MLP 通过多个激活区域形成明显的分段线性（整体非线性）边界

模型	从输入到得分的映射	函数类别
单线性层	$\mathbf{s} = \mathbf{W}^\top \mathbf{x} + \mathbf{b}$	线性
双线性层	$\mathbf{s} = \mathbf{W}_2^\top (\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \tilde{\mathbf{W}}^\top \mathbf{x} + \tilde{\mathbf{b}}$	仍然线性
ReLU MLP	$\mathbf{s} = \mathbf{W}_2^\top \text{ReLU}(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$	非线性

先比较函数类别

前两种模型都能合并为一个线性层

再控制参数量

后两种模型参数量相同，只差 ReLU

结论

接下来观察 ReLU 如何把线性响应 z_j 变为非线性表示坐标 h_j

1 ReLU 门控产生非线性表示

ReLU 门控

$$z_j = \mathbf{w}_j^\top \mathbf{x} + b_j \quad h_j = \text{ReLU}(z_j) = \max(0, z_j)$$

线性中间层

$$h_j = z_j$$

中间层只做线性换坐标
输入空间中仍是固定的线性映射

ReLU 中间层

关闭: $z_j \leq 0 \Rightarrow h_j = 0$ 激活: $z_j > 0 \Rightarrow h_j = z_j$



该单元在短靴样本上产生较大正响应

结论

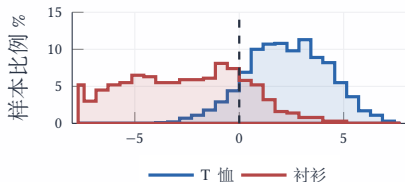
多个 ReLU 单元的开关组合随 $\mathbf{x}$ 改变, 模型在不同输入区域使用不同的线性映射

表示 $\mathbf{h}$ 让线性输出层完成分类

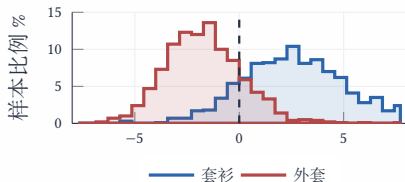
类别得分差

$$\mathbf{h} = [h_1, \dots, h_r]^\top, \quad m_{a,b} = s_a - s_b = (\mathbf{w}_a - \mathbf{w}_b)^\top \mathbf{h} + (b_a - b_b) \leq 0$$

T 恤与衬衫 成对正确率 84.7%



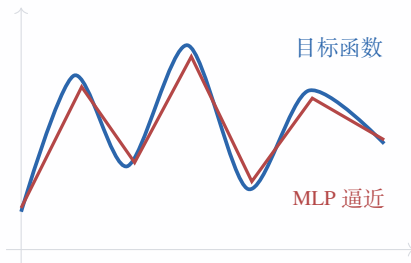
套衫与外套 成对正确率 84.7%



结论

输出层在表示空间中画线性边界；+ 隐藏非线性层，输入空间中边界为非线性

1 能够逼近 $\neq$ 能够训练和泛化



来源: Cybenko 1989

万能逼近定理 Universal Approximation

在适当条件下，足够宽的单隐藏层 MLP 可以在紧集上把连续函数逼近到任意精度

但是，考虑实际应用时：

- 不保证网络很小
- 不保证梯度下降（或其他训练方法）找到目标解
- 不保证有限数据下泛化良好
- 不保证忽略图像结构仍然高效

1 表达能力已经增加，参数怎样“学”出来？



线性层

混合输入证据

ReLU

引入非线性

表示 $\mathbf{h}$

重新组织原始像素

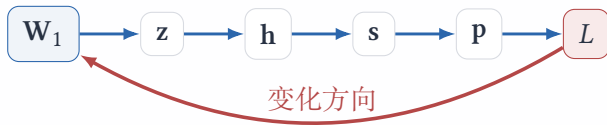
思考

损失怎样把修改方向“传回”每一层参数：反向传播

2

用反向传播训练多层感知机

模型参数为什么会影响最终损失？



梯度

参数发生（局部微小）变化时，最终损失怎样变化

结论

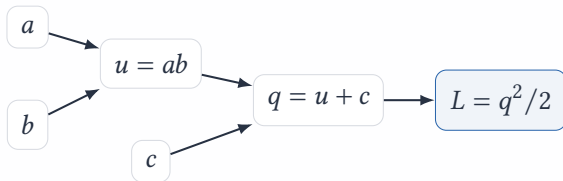
W_1 通过后续节点影响损失 L

2 前向传播可展开为计算图

复合表达式

$$L = \frac{1}{2}(ab + c)^2$$

- 长公式隐藏了依赖顺序
- 局部操作的导数更容易复用



结论

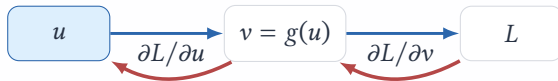
变量是节点，操作连接节点，图保存每个量依赖于谁

2 反向传播只需要局部规则

上游梯度、局部导数和链式法则

若 $v = g(u)$ ，则

$$\frac{\partial L}{\partial u} = \underbrace{\frac{\partial L}{\partial v}}_{\text{上游梯度}} \underbrace{\frac{\partial v}{\partial u}}_{\text{局部导数}}$$



结论

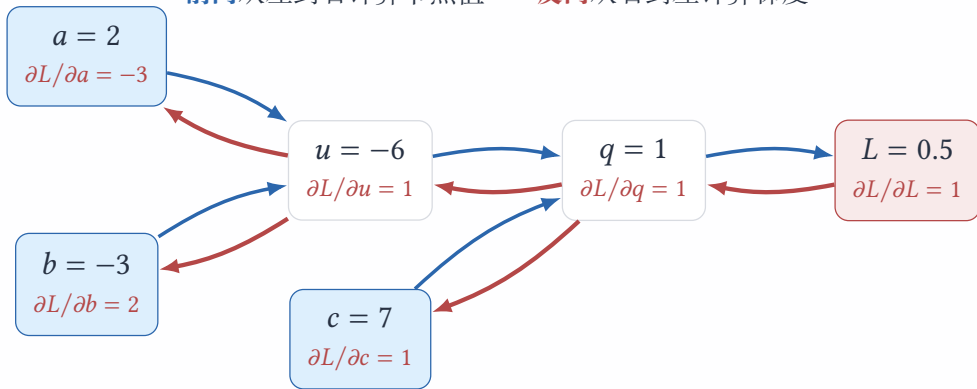
将复杂计算拆解为局部操作，逐一实现

2

同一计算图中的前向与反向传播

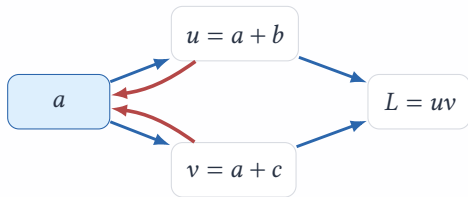
前向 从左到右计算节点值

反向 从右到左计算梯度



节点第一行记录前向值，第二行记录反向梯度

2 分支节点的梯度需要相加



$$\frac{\partial L}{\partial a} = \frac{\partial L}{\partial a} \Big|_{a \rightarrow u} + \frac{\partial L}{\partial a} \Big|_{a \rightarrow v}$$

分支规则

前向时复制数值，反向时累加梯度

定义 | 反向传播

从标量损失出发，按计算图顺序组合“组合”局部导数，得到每个参数的梯度



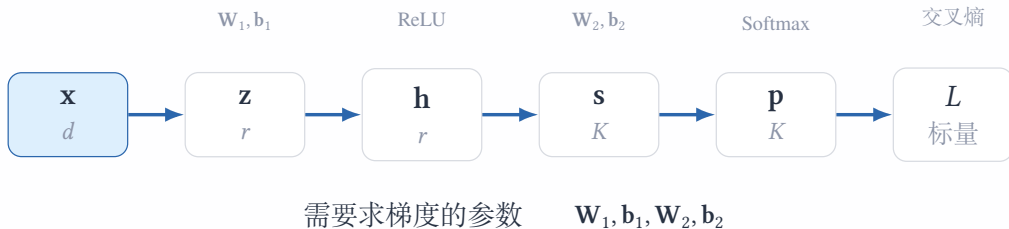
结论

反向传播不是新的求导定理，而是高效复用链式法则的计算方法

2 单隐藏层 MLP 的计算图

从标量到向量与矩阵

局部链式法则不变，节点值、梯度和局部导数变为向量或矩阵



结论

反向传播从 L 出发，沿计算图反方向将梯度“传回”各参数

2 Softmax 交叉熵给出输出梯度



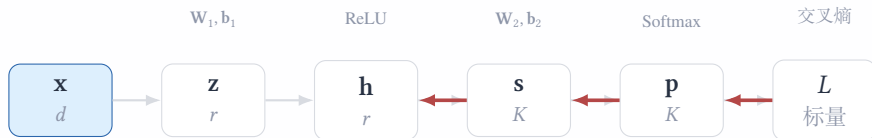
第三讲回顾 | Softmax 交叉熵

$$p_k = \frac{\exp(s_k)}{\sum_{j=1}^K \exp(s_j)} \quad L = - \sum_{k=1}^K y_k \log p_k$$
$$\frac{\partial L}{\partial s_k} = p_k - y_k \quad \delta_s = \frac{\partial L}{\partial \mathbf{s}} = \mathbf{p} - \mathbf{y} \in \mathbb{R}^K$$

结论

正确类别 $y_k = 1$: 梯度为 $p_k - 1$ 其他类别 $y_k = 0$: 梯度为 p_k

2 输出层参数怎样获得梯度？



$$\nabla_{\mathbf{W}_2} L = \mathbf{h} \delta_s^\top \in \mathbb{R}^{r \times K}$$

$$\nabla_{\mathbf{b}_2} L = \delta_s \in \mathbb{R}^K$$

尺寸检查

$$(r \times 1)(1 \times K) = r \times K$$



$$\delta_h = \frac{\partial L}{\partial \mathbf{h}} = \mathbf{W}_2 \delta_s \in \mathbb{R}^r$$

$$\delta_z = \frac{\partial L}{\partial \mathbf{z}} = \delta_h \odot \mathbb{1}[\mathbf{z} > 0] \in \mathbb{R}^r$$

结论

ReLU 是“门控”， $z_j < 0$ 的路径在当前样本上梯度为 0

2 输入层参数怎样获得梯度？



$$\nabla_{\mathbf{W}_1} L = \mathbf{x} \delta_z^\top \in \mathbb{R}^{d \times r}$$

$$\nabla_{\mathbf{b}_1} L = \delta_z \in \mathbb{R}^r$$

尺寸检查

$$(d \times 1)(1 \times r) = d \times r$$

2 一张表总结前向和反向传播

前向	计算	反向	计算
z	$\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1$	δ_s	$\mathbf{p} - \mathbf{y}$
$\mathbf{h}$	$\text{ReLU}(z)$	$\nabla_{\mathbf{W}_2} L$	$\mathbf{h} \delta_s^\top$
$\mathbf{s}$	$\mathbf{W}_2^\top \mathbf{h} + \mathbf{b}_2$	δ_h	$\mathbf{W}_2 \delta_s$
$\mathbf{p}$	$\text{Softmax}(\mathbf{s})$	δ_z	$\delta_h \odot \mathbf{1}[z > 0]$
L	$-\log p_y$	$\nabla_{\mathbf{W}_1} L$	$\mathbf{x} \delta_z^\top$

结论

反向传播按前向计算的相反顺序应用链式法则

损失与输出误差

单样本	$L = -\log p_y,$	$\delta_s = p - y$
mini-batch	$L_B = -\frac{1}{m} \sum_{i=1}^m \log p_{y_i}^{(i)},$	$\Delta_S = \frac{P_B - Y_B}{m}$

反向步骤

单样本

mini-batch (按列堆叠 m 个样本)

输出层参数

$$\nabla_{W_2} L = h \delta_s^\top$$

$$\nabla_{W_2} L_B = H_B \Delta_S^\top$$

$$\nabla_{b_2} L = \delta_s$$

$$\nabla_{b_2} L_B = \Delta_S \mathbf{1}_m$$

传回表示

$$\delta_h = W_2 \delta_s$$

$$\Delta_H = W_2 \Delta_S$$

穿过 ReLU

$$\delta_z = \delta_h \odot \mathbf{1}[z > 0]$$

$$\Delta_Z = \Delta_H \odot \mathbf{1}[Z_B > 0]$$

输入层参数

$$\nabla_{W_1} L = x \delta_z^\top$$

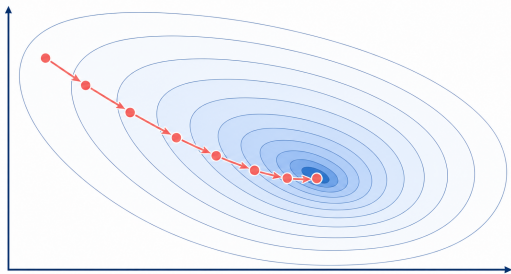
$$\nabla_{W_1} L_B = X_B \Delta_Z^\top$$

$$\nabla_{b_1} L = \delta_z$$

$$\nabla_{b_1} L_B = \Delta_Z \mathbf{1}_m$$

结论

单样本外积在 mini-batch 中变成矩阵乘法, $1/m$ 负责对样本梯度取平均



一次 mini-batch 更新

$$\theta(t+1) = \theta(t) - \eta \nabla_{\theta} L_B$$

$$\theta = \{W_1, b_1, W_2, b_2\}$$

- 其中 L_B 为当前 mini-batch 上交叉熵损失

导数是函数局部的切线斜率

$$\frac{\partial L}{\partial \theta_i} = \lim_{\varepsilon \rightarrow 0} \frac{L(\theta_i + \varepsilon) - L(\theta_i)}{\varepsilon}$$

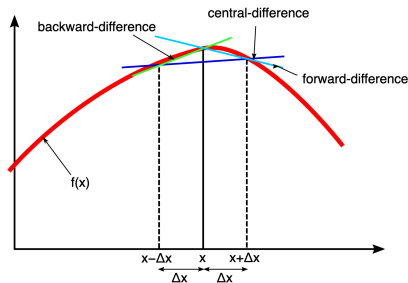
中心有限差分法

$$g_i^{\text{num}} = \frac{L(\theta_i + \varepsilon) - L(\theta_i - \varepsilon)}{2\varepsilon} = \frac{\partial L}{\partial \theta_i} + \mathcal{O}(\varepsilon^2)$$

- 泰勒展开：中心差分有二阶截断误差 $\mathcal{O}(\varepsilon^2)$

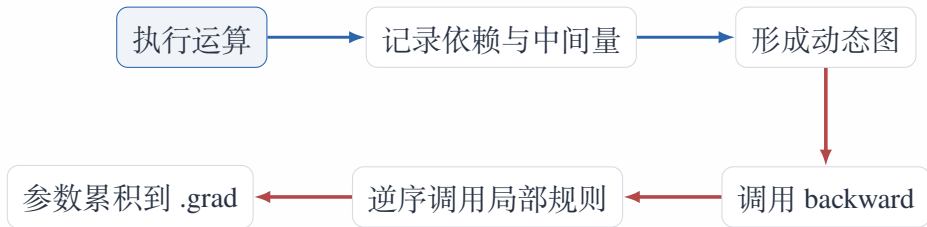
结论

中心差分通过两次前向计算检查反向传播得到的解析梯度



图中 x 对应 θ_i , Δx 对应 ε

来源：Kakitc, Finite difference method, CC BY-SA 4.0



结论

自动微分是一种高效**计算方式**，用来执行链式法则的计算

来源：PyTorch Autograd 官方文档

```

class NeuralNetwork(nn.Module): # 定义模型类
    def __init__(self):         # 创建网络层
        super().__init__()     # 初始化 nn.Module
        self.flatten = nn.Flatten() # 展平图像
        # 按给定顺序连接各层
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512), # 784 维映射到 512 维
            nn.ReLU(),             # 加入非线性
            nn.Linear(512, 512),   # 512 维映射到 512 维
            nn.ReLU(),             # 再次加入非线性
            nn.Linear(512, 10)    # 输出 10 类得分
        )

    def forward(self, x):         # 定义前向计算
        x = self.flatten(x)      # 保留 batch 维
        # 计算每张图像的未归一化类别得分
        logits = self.linear_relu_stack(x)
        return logits            # 返回 logits

```

来源: PyTorch 官方教程, Build the Neural Network

官方模型的单样本公式与维度

$$d = 784, \quad r_1 = r_2 = 512, \quad K = 10$$

$$\mathbf{h}_1 = \text{ReLU}(\mathbf{W}_1^\top \mathbf{x} + \mathbf{b}_1) \in \mathbb{R}^{r_1}$$

$$\mathbf{h}_2 = \text{ReLU}(\mathbf{W}_2^\top \mathbf{h}_1 + \mathbf{b}_2) \in \mathbb{R}^{r_2}$$

$$\mathbf{s} = \mathbf{W}_3^\top \mathbf{h}_2 + \mathbf{b}_3 \in \mathbb{R}^K$$

$$\mathbf{W}_1 \in \mathbb{R}^{d \times r_1}, \quad \mathbf{W}_2 \in \mathbb{R}^{r_1 \times r_2}, \quad \mathbf{W}_3 \in \mathbb{R}^{r_2 \times K}$$

PyTorch 中 `Linear(d,r).weight` 对应 $\mathbf{W}^\top \in \mathbb{R}^{r \times d}$

定义: 得分 logits

最后一层输出 $\mathbf{s} = f_\theta(\mathbf{x}) \in \mathbb{R}^K$ 对应 logits
 $\text{shape}(\text{logits}) = (m, K)$, 它位于 Softmax 之前,
 CrossEntropyLoss 直接接收该张量

```
optimizer.zero_grad()      # 清空上轮累积梯度

logits = model(x)           # 前向计算类别得分
loss = criterion(logits, y) # 计算 mini-batch 损失

loss.backward()             # 反向计算并累积梯度
optimizer.step()            # 根据梯度更新参数
```

1. 清空上一次梯度
2. 前向计算类别得分与损失
3. 反向计算全部参数梯度
4. 优化器更新参数

结论

先清空旧梯度，再执行反向传播

思考

损失不下降时应该先观察什么？

3

模型的训练与泛化





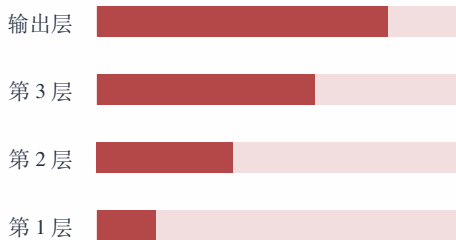
- 损失不降：梯度异常、激活失效或学习率不合适
- 损失震荡：学习率过大或 mini-batch 梯度噪声较强
- 梯度变小：饱和激活、初始化尺度不当或网络过深
- 验证变差：过拟合、预处理不一致或数据分布变化等

结论

控制变量：每次只改变一个因素，判断干预是否有效

$$\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq \frac{1}{4}$$

- 大幅正负输入，导致 Sigmoid 激活进入饱和区
- 多层（链式）相乘持续缩小梯度
- 梯度问题：同时受权重与数据尺度影响

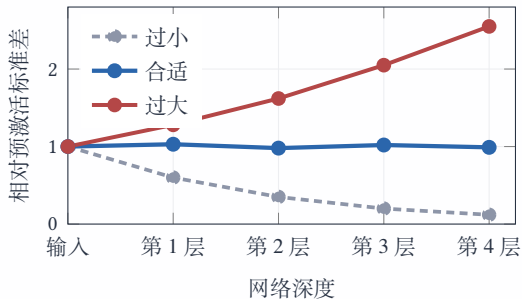


示意 Sigmoid 梯度范数

与 ReLU 对照

ReLU 在正区间的局部导数为 1

初始化尺度影响每层预激活的大小



- 过小：预激活随深度逐层衰减
- 合适：各层保持相近尺度
- 过大：预激活逐层放大或进入饱和区
- 反向传播：梯度也会经历类似的逐层尺度变化

结论

Xavier 与 He 初始化用于稳定深度网络的层间信号尺度

3 从方差传播得到 He 初始化

近似假设：第 l 层输入宽度为 n_{in} ，权重 $W_{l,ij}$ 为零均值且相互独立，并与 $h_{l-1,i} = \text{ReLU}(z_{l-1,i})$ 独立

方差传播计算

$$\begin{aligned} z_{l,j} &= \sum_{i=1}^{n_{\text{in}}} W_{l,ij} h_{l-1,i} \\ \text{Var}(z_{l,j}) &\approx n_{\text{in}} \text{Var}(W_{l,ij}) \text{E}[h_{l-1,i}^2] \\ \text{E}[h_{l-1,i}^2] &\approx \frac{1}{2} \text{E}[z_{l-1,i}^2] \quad \text{ReLU 激活} \\ \text{Var}(z_{l,j}) &\approx \frac{n_{\text{in}}}{2} \text{Var}(W_{l,ij}) \text{E}[z_{l-1,i}^2] \end{aligned}$$

来源：Glorot 与 Bengio 2010, He 等 2015

He 初始化

针对 ReLU 激活函数

$$\text{Var}(W_{l,ij}) \approx \frac{2}{n_{\text{in}}}$$

同 Xavier 初始化对照

近似对称激活函数常用

$$\text{Var}(W_{l,ij}) \approx \frac{2}{n_{\text{in}} + n_{\text{out}}}$$

同时考虑输入与输出宽度

mini-batch 归一化

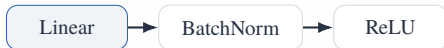
$$\mu_B = \frac{1}{m} \sum_{i=1}^m \mathbf{z}_i$$

$$\hat{\mathbf{z}}_i = \frac{\mathbf{z}_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$\sigma_B^2 = \frac{1}{m} \sum_{i=1}^m (\mathbf{z}_i - \mu_B)^{\odot 2}$$

$$\text{BN}(\mathbf{z}_i) = \gamma \odot \hat{\mathbf{z}}_i + \beta$$

BN 位于 ReLU 前



- 归一化预激活 $\mathbf{z}$
- 原始 BN 与 ResNet v1 采用这一顺序

BN 位于 ReLU 后



- 归一化 ReLU 输出的非负表示 $\mathbf{h}$
- 仿射变换后的输出不必保持非负

结论

普通 MLP 默认采用 Linear、BatchNorm、ReLU

ResNet v2 使用 BatchNorm、ReLU、Conv 的完整预激活 不是右侧顺序的别名

来源: Ioffe 与 Szegedy 2015, He 等 2016 ResNet v1, He 等 2016 ResNet v2, Hasani 与 Khotanlou 2019

训练时

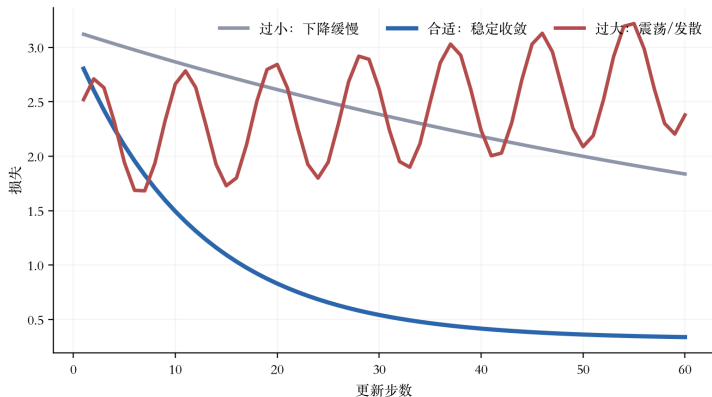
- 使用当前 mini-batch 的均值与方差
- 更新运行均值与运行方差
- 与随机 mini-batch 共同产生噪声

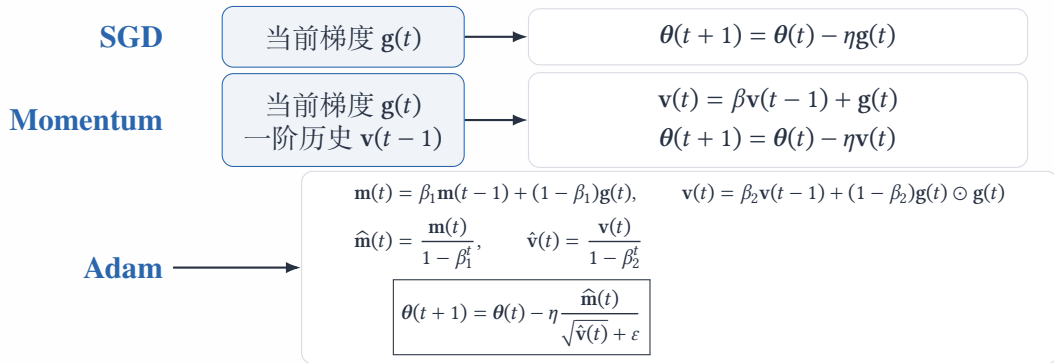
来源: PyTorch BatchNorm1d 官方文档

推理时

- 使用训练期间的统计量
- 不依赖当前测试 batch 的样本
- 保留学习到的 γ 与 β 参数

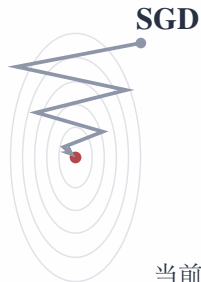
学习率大小：决定训练是收敛还是震荡



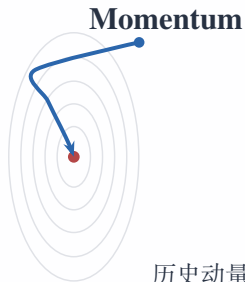


- $\hat{m}(t)$ 给出平滑方向， $\sqrt{\hat{v}(t) + \epsilon}$ 决定各坐标的更新尺度
- 若稳定梯度约为 (10, 0.1)，归一化后两个坐标的更新尺度更接近
- 零初始化会使训练初期的矩估计偏小， $1 - \beta_1^t$ 与 $1 - \beta_2^t$ 用于偏差修正

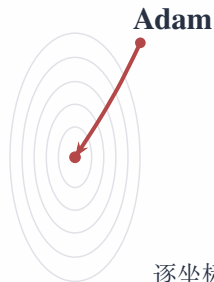
来源：Sutskever 等 2013，Kingma 与 Ba 2015



当前梯度决定方向，狭长方向容易震荡



历史动量平滑方向，横向震荡逐渐减弱



逐坐标缩放步长，不同方向采用不同尺度

Muon 为矩阵参数整形更新方向

矩阵梯度 $G(t)$



动量 $M(t)$



Newton-Schulz



近正交更新 $O(t)$

$$M(t) = \beta M(t-1) + (1-\beta)G(t), \quad M(t) = U\Sigma V^T \implies O(t) \approx UV^T$$

参数分工

- **Muon**: 隐藏层二维权重矩阵
- **AdamW**: 嵌入、输出头、偏置和归一化参数

大模型案例

- **Moonlight**: 3B 激活、16B 总参数的 MoE, 训练 5.7T tokens
- **Kimi K2**: 32B 激活、1T 总参数的 MoE, 训练 15.5T tokens, 使用 MuonClip

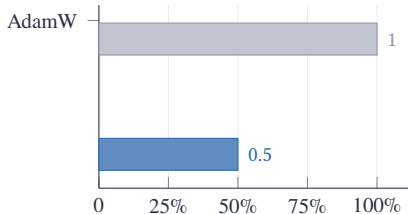
结论

Muon 为矩阵参数提供新的更新几何, 实际大模型训练常与 AdamW 分工使用

来源: Jordan 2024, Liu 等 2025, Moonshot AI Kimi K2

- Transformer 中的参数更新矩阵常由少数奇异方向占据主导
- 若 $G = U\Sigma V^T$, Muon 将更新 $\approx UV^T$
- 抹除奇异值影响: 大奇异值不再继续放大优势, 弱方向获得相对更大更新强度
- Newton-Schulz 迭代避免 SVD 高计算成本

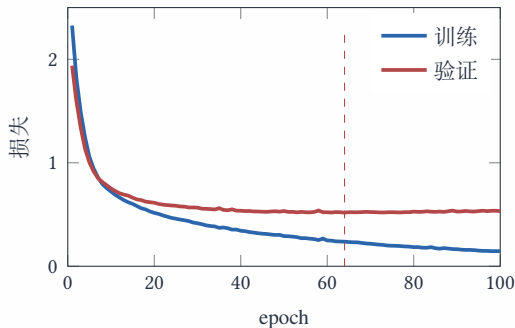
达到相当训练损失的相对计算量



结论

Muon 的目的是**整体**用更少训练 FLOPs 达到目标损失 (不是**单步**优化更便宜)

来源: Jordan 2024, Liu 等 2025



实验设置

- 固定较小训练（子）集
- 使用更宽的 MLP
- 训练足够多轮

- 用 L2 正则、Dropout、数据增强、Early Stopping 等方法减弱或者避免过拟合

3 L2 正则化不总等同于 weight decay

L2 正则化

在优化目标中加入参数惩罚

$$L_{\text{reg}} = L + \frac{\lambda}{2} \|\theta\|_2^2$$

weight decay

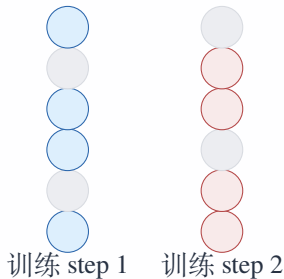
在参数更新中直接缩小参数

$$\theta \leftarrow (1 - \eta\lambda)\theta - \eta g$$

结论

对普通 SGD 等价，对 Adam 等自适应方法优化通常不等价

来源：Loshchilov 与 Hutter 2019



来源: PyTorch Dropout 文档, Srivastava 等 2014

Dropout 实现方法

$$m_j \sim \text{Bernoulli}(1 - p)$$

$$\tilde{h}_j = \frac{m_j h_j}{1 - p}$$

$$\mathbb{E}[\tilde{h}_j] = \frac{\mathbb{E}[m_j]}{1 - p} h_j = h_j$$

- 训练时随机采样掩码 保留单元除以 $1 - p$ 从而保持激活的条件期望
- 推理时关闭 Dropout 并使用完整网络 不再进行额外缩放

```
model = nn.Sequential(                # 按顺序组装 MLP
    nn.Linear(784, 256),              # 计算隐藏单元
    nn.ReLU(),                       # 生成非线性表示
    nn.Dropout(p=0.3),               # 丢弃概率 p
    nn.Linear(256, 10)               # 输出 logits
)

model.train()                        # 开启 Dropout
logits_train = model(x)              # 保留值除以 0.7

model.eval()                        # 关闭 Dropout
with torch.no_grad():               # 不记录梯度
    logits_eval = model(x)          # Dropout 为恒等映射
```

- `train()` 使用随机 mask 与缩放
- `eval()` 不再丢弃也不再缩放

结论

验证与测试前调用 `model.eval()`

来源: PyTorch Dropout 官方文档

3 数据增强和 Early Stopping

数据增强

- 在保持类别语义前提下
- 如：小幅平移、旋转或裁剪等
- 改变训练数据分布



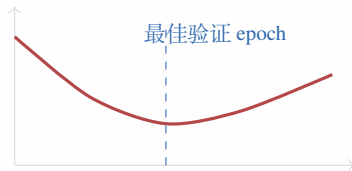
原图



旋转 12° 并平移

Early Stopping

- 用验证集选择停止 epoch
- 保存验证表现最好的参数



设置	验证损失	测试准确率	停止 epoch	主要变化
较宽 MLP 基准	0.533	80.8%	100	无
仅 AdamW	0.532	80.8%	100	解耦权重衰减
仅 Dropout	0.499	81.2%	100	$p = 0.30$
仅轻量增强	0.517	80.8%	100	一像素平移
仅 Early Stopping	0.518	80.5%	64	验证损失选择

结论

不把多种方法叠加后再把改进归因于其中某一种方法

思考

哪一种干预直接针对当前观察到的失败现象？

3 ReLU MLP 是否真正优于两个线性层？

	线性 Softmax	无激活双线性层	ReLU MLP
参数量	7,850	25,450	25,450
验证集选择的学习率	0.003	0.003	0.003
测试准确率	$83.9 \pm 0.1\%$	$84.2 \pm 0.2\%$	$85.5 \pm 0.3\%$
非线性激活	无	无	ReLU

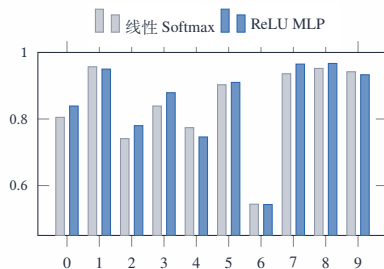
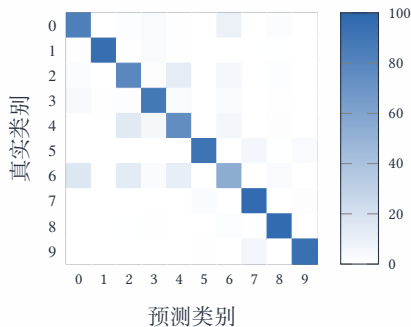
结论

无激活模型即使参数更多，整体函数仍然是线性的

思考

如果 ReLU MLP 的优势稳定出现，最合理的结构解释是什么？

MLP 改善了哪些类别，又留下哪些错误？



- 回顾 $\text{Recall}_k = \frac{\text{TP}_k}{\text{TP}_k + \text{FN}_k}$
- 比较逐类召回率变化
- 查看典型改善与剩余错误

3 MLP 能学习非线性，但没有充分利用图像结构



$28 \times 28 \rightarrow \mathbf{x} \in \mathbb{R}^{784}$

- 输入图像展平后，相邻像素关系没有被保留
- 平移后的同一局部模式需要重新学习，不同位置的相同模式不能自动共享参数
- 参数量随输入尺寸快速增加

思考

怎样让机器学习模型：充分利用数据的空间（局部）结构？

3 从线性模型到可训练的 MLP



- 为什么需要非线性：连续线性层仍可合并，ReLU 改变函数类型
- 梯度怎样跨层传播：反向传播 + 自动微分
- 怎样判断真的更好：固定协议，比较曲线、准确率与错误模式

结论

MLP 让模型学会非线性函数，但仍没有利用图像空间结构

附录导览

- **A | 表达能力与激活函数补充** 万能逼近、激活函数与分段线性区域
- **B | 反向传播与自动微分补充** Jacobian 与反向模式
- **C | 优化方法与曲率** 动量、自适应步长与二阶方法
- **D | 正则化与归一化** L1、L2 与归一化方法
- **E | 实现与实验参考** NumPy、梯度检查与训练循环
- **参考资料** 参考文献与数据、图片、代码来源

附录 A

表达能力与激活函数补充

万能逼近、激活函数与分段线性区域

A 从多层感知机到万能逼近定理

线性阈值单元



隐藏层与激活函数



反向传播联合学习

Rosenblatt 1958

Rumelhart 等 1986

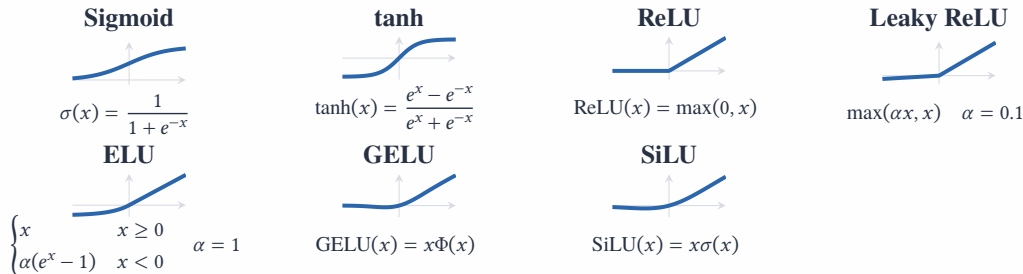
存在性结论

对紧集上的连续函数 f ，在适当激活条件下，对任意 $\varepsilon > 0$ ，存在足够宽的单隐藏层网络 g 使

$$\sup_{\mathbf{x} \in \mathcal{X}} |f(\mathbf{x}) - g(\mathbf{x})| < \varepsilon$$

- 定理给出：存在某类网络具有逼近能力
- 定理没有给出：如何得到这类网络的高效结构、训练算法与泛化保证

A 常见激活函数的图像与表达式

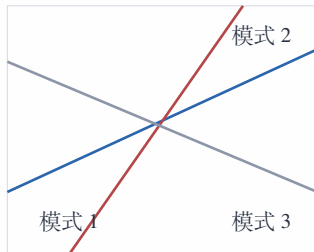


- Sigmoid 常用于二分类输出层与门控；tanh 常用于循环网络；二者输出有界且平滑，但大幅度输入容易饱和并使梯度变小
- ReLU 是 MLP 与 CNN 隐藏层的常用基线，计算简单且正区间不饱和，但负区间可能产生长期关闭单元
- Leaky ReLU 与 ELU 保留负区间梯度，可缓解关闭单元问题，代价是额外超参数或指数计算
- GELU 常见于 BERT 与 ViT；SiLU 常见于 EfficientNet；曲线平滑且允许小负输出，但计算开销高于 ReLU 且效果依赖具体结构

A ReLU 网络为何产生分段线性区域？

$$f_{\theta}(\mathbf{x}) = \mathbf{A}_r \mathbf{x} + \mathbf{b}_r \quad \mathbf{x} \in \mathcal{R}_r$$

- 每个 ReLU 只有开与关两种局部状态
- 固定激活模式后网络在区域内是线性/仿射函数
- 不同激活模式把输入空间切分成凸多面体区域



延伸阅读

Hinz 与 van de Geer 逐层追踪区域及其像空间维度，给出 ReLU 网络仿射线性区域数量的统一上界；核心启示是深度可反复细分给定的输入空间

Hinz and van de Geer, A Framework for the Construction of Upper Bounds on the Number of Affine Linear Regions of ReLU Feed-Forward Neural Networks

IEEE Transactions on Information Theory, 65(11):7304–7324, 2019 doi:10.1109/TIT.2019.2927252

附录 B

反向传播与自动微分补充

Jacobian 与反向模式

B Jacobian 矩阵、 反向模式计算向量—Jacobian 乘积

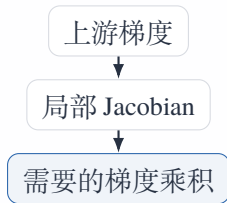
Jacobian

对 $\mathbf{v} = g(\mathbf{u})$, $\mathbf{v} \in \mathbb{R}^m, \mathbf{u} \in \mathbb{R}^n$

$$\mathbf{J}_{\mathbf{v}, \mathbf{u}} = \frac{\partial \mathbf{v}}{\partial \mathbf{u}} = \begin{bmatrix} \partial v_1 / \partial u_1 & \cdots & \partial v_1 / \partial u_n \\ \vdots & \ddots & \vdots \\ \partial v_m / \partial u_1 & \cdots & \partial v_m / \partial u_n \end{bmatrix} \in \mathbb{R}^{m \times n}$$

$$\frac{\partial L}{\partial \mathbf{u}} = \mathbf{J}_{\mathbf{v}, \mathbf{u}}^\top \frac{\partial L}{\partial \mathbf{v}}$$

- 标量损失只有一个输出方向
- 参数数量可以非常大
- 无需显式保存完整 Jacobian, 只保存矩阵向量乘积



附录 C

优化方法与曲率

动量、自适应步长与二阶方法

C Momentum 与 Nesterov 的完整更新

Momentum

$$\mathbf{v}(t+1) = \mu \mathbf{v}(t) - \eta \mathbf{g}(t)$$

$$\boldsymbol{\theta}(t+1) = \boldsymbol{\theta}(t) + \mathbf{v}(t+1)$$

Nesterov

$$\tilde{\boldsymbol{\theta}}(t) = \boldsymbol{\theta}(t) + \mu \mathbf{v}(t)$$

$$\mathbf{g}(t) = \nabla L(\tilde{\boldsymbol{\theta}}(t))$$

结论

Nesterov 在沿动量前视的位置计算梯度

C AdaGrad 与 RMSProp 怎样改变步长？

AdaGrad

$$\mathbf{v}(t) = \mathbf{v}(t-1) + \mathbf{g}(t) \odot \mathbf{g}(t)$$

平方梯度持续累积，步长单调减小

RMSProp

$$\mathbf{v}(t) = \rho \mathbf{v}(t-1) + (1 - \rho) \mathbf{g}(t)^2$$

指数移动平均缓解持续衰减

结论

两种方法都按参数维度使用历史梯度尺度调整更新

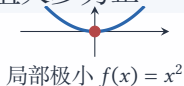
C Hessian 描述局部曲率

Hessian 矩阵

$$H(\theta) = \nabla_{\theta}^2 L(\theta)$$

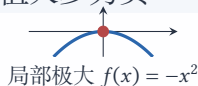
局部极小

特征值大多为正



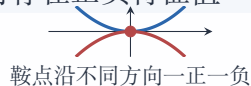
局部极大

特征值大多为负



鞍点

同时存在正负特征值



结论

神经网络的 Hessian 规模巨大并且通常可能不定 (i.e. 存在零或者负特征值)

C 二阶方法为什么没有直接取代一阶方法？

Newton 法

$$\theta(t+1) = \theta(t) - \mathbf{H}(t)^{-1} \mathbf{g}(t)$$

- Hessian 计算和存储代价高
- mini-batch 曲率带有噪声，导致估计 Hessian 可能不定

结论

实际系统常使用一阶方法或近似曲率信息，如 Hessian 的对角矩阵近似

附录 D

正则化与归一化

L1、L2 与归一化方法

D L1 与 L2 正则化改变参数收缩方式

L1 范数

$$\|\boldsymbol{\theta}\|_1 = \sum_{j=1}^d |\theta_j|, \quad L_{\text{reg}} = L + \lambda \|\boldsymbol{\theta}\|_1$$

- 在零点不可微，软阈值能将小坐标精确变成零，从而形成稀疏参数

L2 范数

$$\|\boldsymbol{\theta}\|_2 = \sqrt{\sum_{j=1}^d \theta_j^2}, \quad L_{\text{reg}} = L + \frac{\lambda}{2} \|\boldsymbol{\theta}\|_2^2$$

- 平方惩罚处处可微，各坐标按比例连续收缩，通常变小但仍保持非零

具体例子：对同一向量施加一次正则化收缩

设更新前 $\mathbf{u} = (1.4, 0.5, 0.2)$ 且收缩强度 $\tau = 0.4$

$$\underbrace{\text{sign}(\mathbf{u}) \max(|\mathbf{u}| - \tau, 0)}_{\text{L1 软阈值}} = (1.0, 0.1, 0),$$

$$\underbrace{\frac{\mathbf{u}}{1 + \tau}}_{\text{L2 连续收缩}} \approx (1.00, 0.36, 0.14)$$

D Batch Normalization 的动机与完整算法

训练中的尺度变化

- 前层参数更新会改变后层预激活的均值与尺度
- 尺度过大或过小时，激活可能饱和，梯度也可能逐层放大或缩小
- 因此，大规模深层网络更依赖合适的初始化与学习率
- 核心问题：优化稳定性
- BN 可以控制特征尺度，使梯度变化可控、可预测，支持较大学习率

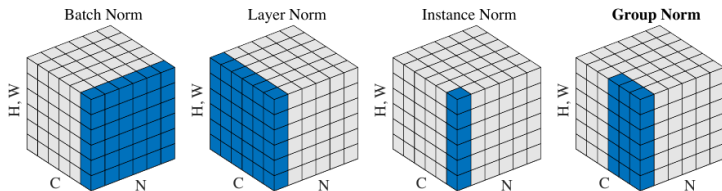
来源：Ioffe 与 Szegedy 2015, Santurkar 等 2018

对 mini-batch 中每个特征归一化

$$\begin{aligned}\mu_B &= \frac{1}{m} \sum_{i=1}^m z_i, & \sigma_B^2 &= \frac{1}{m} \sum_{i=1}^m (z_i - \mu_B)^2 \\ \hat{z}_i &= \frac{z_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, & y_i &= \gamma \hat{z}_i + \beta\end{aligned}$$

- 训练时使用 batch 统计量并更新滑动均值与方差
- γ, β 保留可学习的尺度与偏移 推理时使用保存的统计量

D 其他归一化方法使用哪些统计量？



蓝色单元共享同一组均值与方差 图中 N 为 batch 轴 C 为通道轴 H, W 为空间轴

方法	归一化范围	是否依赖 batch	常见场景
BatchNorm	同一通道的 N, H, W	是	较大 batch 的视觉模型
LayerNorm	单样本的 C, H, W	否	Transformer 与序列模型
InstanceNorm	单样本单通道的 H, W	否	风格迁移与图像生成
GroupNorm	单样本分组通道的 C, H, W	否	小 batch 视觉模型

对比图来源：Wu 与 He 2018 Figure 2 参考：BatchNorm, LayerNorm, InstanceNorm, GroupNorm

附录 E

实现与实验参考

NumPy、梯度检查与训练循环

E NumPy forward/backward 最小实现

```
z = x @ W1 + b1
h = np.maximum(z, 0)
s = h @ W2 + b2
p = softmax(s)

ds = p - onehot(y)
dW2 = h.T @ ds / m
dh = ds @ W2.T
dz = dh * (z > 0)
dW1 = x.T @ dz / m
```

- forward 返回损失与缓存
- backward 读取缓存
- 每组参数返回同形状梯度
- 偏置梯度沿 batch 维求和

```
for index in sampled_parameter_indices:
    old = theta[index]

    theta[index] = old + eps
    loss_plus = forward(theta)

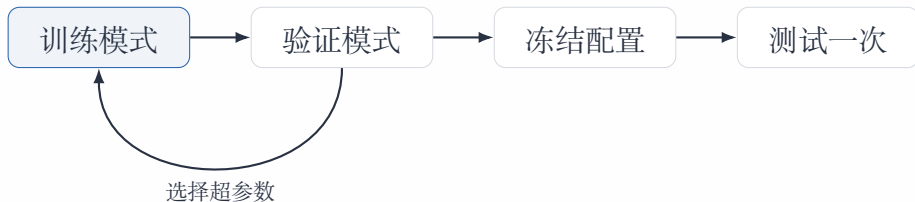
    theta[index] = old - eps
    loss_minus = forward(theta)

    theta[index] = old
    grad_num = (loss_plus - loss_minus) / (2 * eps)
    compare(grad_num, grad_backward[index])
```

结论

对每组参数分别报告最大相对误差并固定随机操作

E PyTorch 完整训练与评价循环



训练阶段

`DataLoader`、`zero_grad`、`forward`、`backward`、`step`

评价阶段

`model.eval`、`no_grad`、保存最佳
checkpoint

E 可复现实验需要记录什么？

- 数据版本与划分
- 模型结构与参数量
- 随机种子与初始化
- 优化器与学习率
- batch size 与最大 epoch
- Early Stopping 规则
- 软件与硬件版本
- 代表性种子规则
- 均值与标准差
- 原始曲线和配置文件

结论

保留记录，所有核心结果都可以从原始数值重新生成

参考文献

来源与参考文献

研究论文

- [1] Xiao, Rasul, Vollgraf 2017 Fashion-MNIST
- [2] Rumelhart, Hinton, Williams 1986 Back-propagation
- [3] Cybenko 1989 Universal approximation
- [4] Glorot, Bengio 2010 Xavier initialization
- [5] He 等 2015 Rectifiers and initialization
- [6] Ioffe, Szegedy 2015 Batch Normalization
- [7] Santurkar 等 2018 How Does BN Help Optimization
- [8] Srivastava 等 2014 Dropout
- [9] Kingma, Ba 2015 Adam
- [10] Loshchilov, Hutter 2019 AdamW
- [11] Prechelt 1998 Early Stopping
- [12] Ba, Kiros, Hinton 2016 Layer Normalization
- [13] Ulyanov 等 2016 Instance Normalization
- [14] Wu, He 2018 Group Normalization

论文标题均可点击；完整网址与逐页用途见同目录 SOURCES.md

数据、图片与代码来源

来源与参考文献

数据与工具

- **Fashion-MNIST**
Zalando Research 官方仓库
- **Half-moon**
scikit-learn `make_moons` 文档
- **自动微分**
PyTorch Autograd 文档

图片与代码

- **机制图**
本讲 LaTeX 与 TikZ 原创绘制
- **实验图**
本讲实验脚本从官方数据与保存数值生成
- **复用素材**
第三讲已记录来源的 Fashion-MNIST 本地生成资产及学习率教学示意图

实验记录：参数与结果见 `assets/generated/experiment-metadata.json`；学习率教学示意图复用第三讲，不代表本讲 MLP 的实测结果